忆时塔（Chronomem）：以 MCP 服务形态提供的时间分层外部记忆——回想与语义总结

YSTYLE[1] · ORCID：0009-0009-6929-0070 · lxy5266@live.com
DeepSeek V4.1 Flash[2] · AI 共同作者

[1] *position / 设计论文，尚无实证评估。实现与完整技术设计见* `https://atomgit.com/ystyle/chronomem` *（含记忆服务本体与* `dsh-mem-bridge/` *——自动记录 Agent 会话活动的宿主插件，即 §6 活动语料的来源）。本文为中文审阅版；**投稿以英文版为准**。* [2] *AI 共同作者（与 AiraXiv 预印本的署名一致）。详见文末 AI 使用披露。*

摘要

长周期运行的 Agent（跨周、跨月地持续工作：迭代一个代码库、写一篇论文、长期维护一个项目）需要一个根本性的能力：记住很久以前发生的事。但今天的 Agent 外部记忆系统要么是「扁平存储」（全存下来、按近因/相似度检索），要么是「一次性压缩」（定期把整段对话压成一份报告）。前者不提供任何压缩：过了几个月，没有任何一层能告诉你「这一阵子大概发生了什么」；后者破坏证据：决定被记住了，但「为什么、怎么做的」——那些原始事件——丢了，且无法重新推导。

忆时塔（Chronomem，「记忆时间之塔」）旨在占据中间地带，我们称之为**时间分层记忆（time-layered memory）**：把事件记录组织成一张**随时间分层的物化快照层级**——从 10 分钟到一周——每一层既是(a)指向更细层、最终指向原始记录的**导航结构**，又是(b)可一眼读懂的**可选语义摘要**。这模仿两种日常机制：**财务汇总报表**（原始账本永不删除、报表渐进重算、每张报表都能审计回底层分录）与**日报体系**（任务记录逐层汇总成日报、周报、月报，越往上越简洁）。

我们把忆时塔做成一个 **MCP（Model Context Protocol）服务**——一个常驻守护进程，任何支持 MCP 的客户端（opencode、Cherry Studio、自定义 Agent 等）都能接入。我们主张：把记忆做成**协议**而非私有库，本身就是一个系统层面的贡献——它让异构工具在零代码耦合的前提下共享同一个记忆底座。

本文是一篇 **position（定位）与设计**论文：我们从两种日常的分层汇总实践（财务汇总报表与日/周/月报体系）出发论证设计动机，把它与当前 LLM 记忆系统的相关工作对齐，陈述区分本方案的双轨成本模型，并列出计划用实证回答的开放问题。

1. 引言

Agent 在长时间尺度上工作，遇到的根本限制是：上下文窗口有界，而值得记住的事件无界。现有方法分成两派，各自有相反的失败模式：

- **扁平外部记忆**（全存，按近因或相似度取回）：保留了细节，但没有压缩——几个月后，没有任何层级能一句话告诉你「这段时间发生了什么」，除非把全部内容读一遍。
- **一次性压缩**（定期把整个会话压成一份报告）：破坏了证据——决定被记住了，但「为什么、为什么选它、怎么做的」这些原始事件没了，且无法重新推导。

忆时塔的设计目标正是这两者的中间态。在时间上分层、物化：每一层同时是「导航结构」（引用更细层直到原始记录）和「可选语义摘要」（一眼可读）。这与两种日常机制同构：**财务汇总报表**（账本永不删，报表

渐进算，报表可审计）与**日报体系**（任务 → 日 → 周 → 月，越远越简洁）。

我们将忆时塔暴露为 **MCP 服务**——一个常驻守护进程，任何 MCP 客户端都能接入。把记忆做成**协议**而不是私有库，我们认为是系统层面的贡献：让异构工具共享一个记忆底座，而不需要代码耦合。

本文定位为**设计/观点论文**：从两种日常的分层汇总实践（财务汇总报表与日/周/月报体系）出发论证设计动机，与 LLM 记忆系统的工作对齐，陈述「双轨成本模型」这一核心主张，并列出拟用实证回答的开放问题。

2. 背景与相关工作

日常实践中的分层汇总。 有两种日常机制靠「逐层向上汇总」运作，它们直接启发了本设计：**日报体系**（任务记录 → 日报 → 周报 → 月报，每一层都比下一层更简洁）与**财务汇总报表**（原始账本 → 定期报表；账本永不删除、报表渐进重算、每张报表都能审计回底层分录）。二者共有的、也是我们借用的性质是：明细完整保留，而每一层都能一眼回答「这个尺度上发生了什么」。忆时塔把这个机制工程化到 Agent 记忆上：细粒度记录保留，每个粗粒度层同时是指向更细层的引用链与可读的语义摘要。这一结构也与认知科学里记忆巩固（consolidation；睡眠相关的重放 replay）的直觉相呼应，但我们不主张任何生物意义上的保真。从报表实践迁移过来的，刻意是**纪律**而非聚合语义——明细层不可变、可审计的下钻、幂等的渐进重算；这一迁移来自作者在 ERP 与报表系统上的工程实践。与 ERP 的合并报表不同，那里的汇总是对结构化分录、按法定固定日历做确定性求和，而这里的每一层要用 LLM 总结任意 JSON 事件与自然语言记录，粒度链按记忆类型可配，且导航与压缩之间的成本差异成为首要的设计变量。

时序数据库与 rollup。 分层时间窗 + 物化聚合在时序数据库中是标配（rollup / 连续聚合 / 降采样，如 TimescaleDB 连续聚合、Prometheus 降采样）。忆时塔复用了这套机制（基于 `tsdb` 库），但向数据库不做的方向扩展了一层：粗层不仅是数值聚合，还可以是**对任意 JSON 事件生成的 LLM 语义摘要**。

LLM 记忆系统。 为 LLM Agent 提供长期记忆的研究正在快速增长：MemGPT（Packer 等）引入分层上下文管理与记忆层级；Generative Agents（Park 等）把「记忆流」与检索、反思结合；A-MEM〔arXiv:2502.12110〕与 Mem0〔arXiv:2504.19413〕发展出 agentic / 组织化的记忆；**HiMem**（arXiv:2601.06377）为长周期 Agent 提出分层长期记忆；**RaMem**（arXiv:2606.22844）研究情境重激活；**AriadneMem**（arXiv:2603.03290）处理 Agent 的终生记忆。我们的差异点在于：(i) 把「精确回想」（走引用链、服务端不调 LLM）与「语义回想」（LLM、压缩）作为**显式的成本模型**分开；(ii) 摘要是**一等公民的时序条目**，可被递归再总结；(iii) 把整个底座标准化在 MCP 接口后面。

主题记忆 vs 时间记忆。 「LLM wiki」一类系统按主题/语义词条组织知识——主题记忆，类比维基百科。这个范式——把来源**编译**成一个由 LLM 持续维护的持久 wiki，而不是在查询时检索原始片段——由 Karpathy 提出〔Karpathy，《LLM Wiki》，2026〕，此后引来设计与生产层面的分析（*Beyond RAG: LLM Wikis as Living Semantic Memory*〔Zenodo 20078453〕；*Compiled Memory vs Governed State: Karpathy's LLM Wiki and the Reflexive Laboratory*〔Zenodo 19507593〕；*Managed Memory Is Still Not Governed State: Production Lessons from LLM Wiki v2*〔Zenodo 19545947〕）。忆时塔按时间组织——时间记忆，类比账本。二者互补：时间日志可沉淀出主题词条，词条可回溯到时间证据。我们把这个对比讲清楚，是因为我们相信很多「记忆」系统混淆了两条轴，而轴的选择应跟随用途：论文是「过程」（时间），参考手册是「知识体」（主题）。

Model Context Protocol。 MCP（modelcontextprotocol.io）标准化了客户端与服务器之间的工具、资源与提示。忆时塔在此的贡献是：通过 `mem/*` 工具集把**记忆**做成一等 MCP 能力，让任何 MCP 客户端无

需集成私有 SDK 即可采用共享、演化的记忆。（目前只以工具形式暴露记忆；资源与提示是同一底座上自然的后续扩展。）

3. 设计总览

忆时塔是一个独立的守护进程，只提供一个 streamable-HTTP MCP 端点（启用 Web 控制台时与 MCP 共用同一端口，MCP 挂在 `/mcp`）。存储是单一时序数据库（`tsdb`），每个记忆类型占两个 series——原始条目一个、其摘要一个。设计有五大支柱：

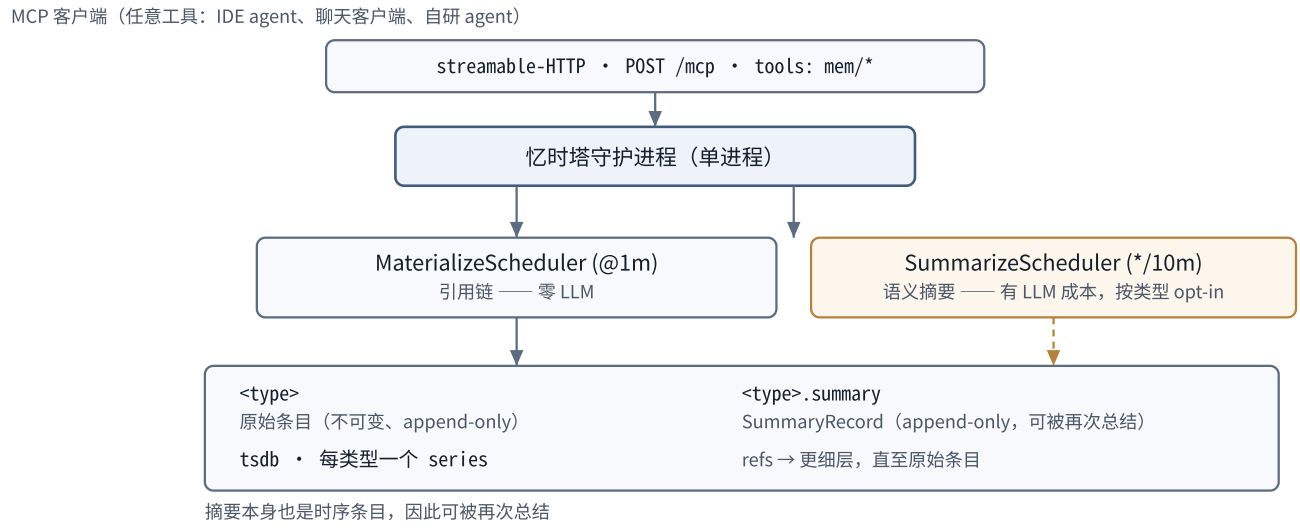


图 1. 架构。客户端经 MCP 接入单一守护进程；两个独立调度器分别物化一个类型的两个 series——引用链（零成本、分钟级）与语义摘要（有 LLM 成本、按类型 opt-in）。摘要存于自己那条 append-only series，并通过 refs 指向更细层，这正是递归总结得以成立的原因（§3 P5）。

P1——记忆类型由用户自定义。 一个*类型*是一条带名字的 series，含可选描述与 schema（供文档与 LLM 提示用，不强制校验）、一个已声明的保留期字段（当前仅为元数据，MVP 不做淘汰），以及*粒度链*（见下）。客户端可建 `work/chat`、`paper/decisions`、`tool/errors` 等类型。类型名白名单（`[A-Za-z0-9_/-]`），防路径穿越。

P2——类型级粒度链。 每个类型声明自己的物化层级，从 {10min, 30min, 1h, 3h, day, week, month} 中取，默认为 10min→30min→1h→day→week。高频通道保留分钟级层级以支持**当天内精确回想**；稀疏通道（如一天一条的工作日志）用 day→week，避免建大量近乎为空的细桶。week 是默认的语义总结封顶：一整年的周级摘要只有约 52 条，加载便宜，且整年回忆极少需要。

P3——物化模式。 一个塔层原则上可以用三种方式物化，成本不同。当前实现交付前两种。第三种**尚未实现**：底层存储只提供一组最小的聚合原语（计数、最新、均匀抽样），要支撑指标型类型的零 LLM 概览层，还需先补上通用数值聚合（均值/最小/最大/分位数）。

模式	塔层内容	支持的回想	LLM 成本	状态
引用链（默认，事件型）	`PointRef` / `BucketRef` 指向更细层	精确下钻	零	已实现
语义摘要（可选，LLM）	LLM `SummaryRecord`	语义回顾	有	已实现
数值聚合（指标型）	count/avg/min/max JSON	概览 + 下钻	零	规划中（当前仅有 count/latest/sample 原语）

P4——双轨回想。 两个读操作，成本差异明确：

- `mem/recall mode=exact`：沿引用链走到**原始条目**——无损、服务端不调 LLM，但返回的是原始体量（引用链是**导航**，不是**压缩**；客户端要为注入这份体量付费）。
- `mem/recall mode=summary`：返回窗口的**语义摘要**——这才是真正压缩粗窗口的形态，且**依赖摘要已生成**（此前经 `mem/summarize`，或 `summarize_on_miss=true` 时按需现算）。

P5——摘要是时序记录的一等公民。 `mem/summarize` 读一个窗口（按粒度+锚点，或按显式 `from/to` 处理不规则的「任务阶段」窗口），让 LLM 生成结构化摘要（锚点章节：决策/结论/待办/状态变更/关键词），并追加一条 `SummaryRecord` 到该类型的 `.summary` series：核心字段 `{ts, windowFrom, windowTo, model, refs, instructions, text, groupKey}`，另有生成口径字段 `{strategy, chunks, inputTokens, clippedEntries}`——记录该摘要是一次直连产出还是分块 map-reduce 聚合，以及为预算截断了多少条目。因为摘要本身是时序条目，可被再次总结：原始 → 10min 摘要 → 1h 摘要 → day 摘要，就是结构塔的语义对应物，实现**递归、非破坏的压缩**。具体地，总结**默认自底向上**进行：在总结一个粗窗口之前，服务会先确保它内部每个更细子窗口都有摘要（缺失的先按原始条目补出来），再以这些子摘要聚合——于是粗粒度总结的成本是「每个分组一次调用」，而不是把整个窗口重读一遍，而 `refs` 会记录材料实际取自哪一层。想彻底避开链式转述的调用方，可用 `recompute` 绕开递归。

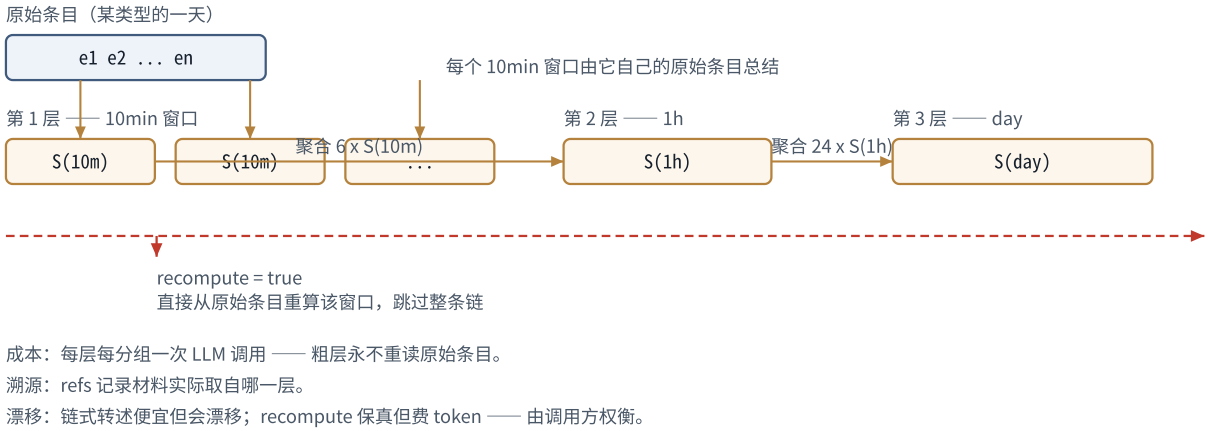


图 2. 自底向上的递归总结（§3 P5）。每一层由它下面一层总结而来；更细层缺失时按需现算；`recompute` 绕过链条、直接从原始条目重建窗口。每条摘要本身都是 append-only，链条上没有任何东西被销毁。

4. 双轨成本模型

我们的核心概念主张是：**回想不能被当成单一操作**。要区分：

- **精确回想 = 服务端不调 LLM 的导航。** 服务把窗口沿物化引用链解析成原始条目，全程不调用 LLM。可靠、无损，但信息量就是原始体量；粗层的精确回想会把全部内容都取回来，不提供任何压缩收益。这是**导航**，不是**总结**。
- **语义回想 = 读摘要。** 这是唯一真正满足「窗口越粗、答案越小」的形态。但它有一个硬**前提**：摘要必须存在（之前生成过，或现在生成、花 LLM 成本）。因此，「回想不花 LLM token」**只在服务端一侧、且只对精确下钻成立**；粗粒度概览式回想本质上是依赖总结的。

token 账是双侧的，「零 token」只是其中一侧的说法。 我们把两侧分开：

	服务端	客户端
精确回想	不调 LLM	**返回的原始条目要注入客户端 prompt —— 这是真实 token 成本，且随体量增长**
语义回想	生成/缓存摘要时调 LLM	注入摘要文本 —— 更小，但仍非零
结构物化	不调 LLM	无（客户端根本看不到桶）

所以「精确回想免费」只对**服务端**成立：它确实从不调用模型。但它**并非端到端免费**——导航是用体量换保真，而体量正是客户端要花 token 塞进上下文窗口的东西。这恰恰是双轨模型要暴露的取舍：精确回想把 token 成本转成**客户端** token 并保留证据；语义回想把它转成**服务端** token 并丢失细节。部署要优化的是 `服务端 + 客户端` 的总和，而不是其中任一侧。

这个区分回过头看很平凡，但我们认为现有系统把它模糊了，并由此产生三个具体后果：

1. **触发策略。** 结构物化（引用链、聚合）是每分钟跑、零 token 的作业。语义总结正相反：没有护栏的调度器会在空窗口或低价值窗口上悄悄烧 token，还会对注定失败的窗口反复重试。因此忆时塔把它做成**按类型 opt-in**（新建类型默认关闭），一旦开启则用五道护栏约束它：(a) 只总结**最近一个已闭合**窗口，且有沉降期——窗口仍在被写入时绝不总结；(b) 按窗口、按分组幂等，成功即清空重试账；(c) 空窗口零成本、直接跳过；(d) 连续失败会退避，到达上限后登记为「已放弃」并留痕，而不是每十分钟永远重试；(e) 每轮提交数有上限，积压不会变成无界突发。内置的 `worklog` 类型开箱即启用（每个已闭合的自然日、每个项目一条），任意新建类型默认关闭；回想时的「摘要缺失」路径仍是显式的（`summarize_on_miss`）。
2. **语义漂移。** 对摘要再总结很便宜但会漂移；从原始数据重算保真但昂贵。把两者都暴露出来、用显式的 `recompute` 切换，把一个静默的质量问题变成调用方可控的权衡。
3. **上下文卫生。** 服务只存和取，从不主动把内容塞回客户端上下文。是否取回、取回多少，由客户端决定，从而守住「恒定上下文」的纪律。

5. 开放问题

- **语义漂移的量化。** 摘要链多久会偏离底层事件？什么时候 `recompute` 值回 token？我们计划用一组固定的探针问题测量沿链的信息保持度。
- **受 token 预算约束的触发。** 给定 token 预算，摘要应该主动算（如每小时/每周末）还是按需惰性算？这是关于 {窗口, 价值, 成本} 的优化问题。

- **保留与隐私**。 MVP 全量存 payload、无隐私分级；类型级隐私/加密是后续工作——动机是记忆越积越敏感。
- **评估方法论**。 Agent 记忆评估还不成熟。我们提议：固定长周期任务套件、带真值答案的查询集、指标{回想精确率/召回率、token 成本、时延}，对比基线{扁平近因记忆、一次性压缩、MemGPT 式记忆层级}。运营自有部署给我们的一个教训是：**物化视图的一致性应当作为指标，而不是默认成立的前提**——分层存储可能返回陈旧窗口（该窗口最后一次物化之后写入的条目直接缺失），或干脆返回相邻窗口的记录，而返回的证据看上去完全自洽、不带任何错误信号。因此对分层记忆的评估应当包含一项显式的一致性核对：每个粗粒度视图与它声称覆盖的原始记录之间是否吻合。

6. 评估计划（待进行）

实证结果超出本文 position 论文的范围，将出现在后续论文中。计划中的研究：(i) 受控的长周期 Agent 任务（如跨模拟数周递增维护项目），对比忆时塔与扁平记忆、一次性压缩在回想准确率与 token 成本上的表现；(ii) 沿摘要链的语义漂移研究；(iii) 由作者开展的工作日志用例（day/week 粒度 + 任务阶段总结）定性研究。作者自 2026 年 8 月起持续运行一套单用户部署；截至 2026 年 10 月，库中已有 450 条以上结构化工作日志、1000 条以上 Agent 活动记录，以及 100 条以上按日/周递归生成的摘要；这批语料将作为 (iii) 的素材，并为校准 (i)(ii) 提供真实的窗口规模与压缩比分布。

7. 实现状态

为免「已实现」与「设计中」混淆，逐一标注：

组件	状态	说明
记忆类型、类型级粒度链	**已实现**	运行时动态，新增类型无需改码
日历对齐的自然窗口（day/week/month）	**已实现**	时区可配
引用链物化 + `exact` 下钻	**已实现**	默认层；服务端零 LLM
语义摘要 + 递归（自底向上）总结	**已实现**	按类型 opt-in；`recompute` 可绕过链条
带护栏的定时自动总结	**已实现**	沉降 / 幂等 / 退避 / 每轮上限
MCP 工具面（`mem/*`）	**已实现**	13 个工具；streamable HTTP
内嵌 Web 控制台	**已实现**	与 MCP 同端口
会话活动自动采集	**已实现**	以宿主插件形式随本仓分发
数值聚合（指标型）	**规划中**	当前只有 count/latest/sample 原语
保留期 / 淘汰	**规划中**	类型元数据里可声明，但不生效；实际全量保留
MCP resources	**规划中**	目前只有 tools
鉴权、加密、多租户隔离	**规划中**	当前为单用户、仅本机部署
实证评估	**规划中**	见 §6

8. 局限与边界

- **这是工程底座，不是新的摘要算法。** 我们主张的新颖性在于成本感知的 API 面、append-only 的递归摘要、以及 MCP 暴露——而非任何单一组件（每个都借自他处）。
- **本文不含实证评估。** 关于 token 节省、漂移与运行成本的论断是论证出来的，不是测量出来的。§6 给出计划；§5/§6 引用的运行数字是部署观察，不是受控实验结果。
- **没有基线与消融。** 与替代架构的对比只停留在定性层面。
- **部署是单用户、仅本机的。** 多租户、鉴权、加密运行都未经验证；§5 的隐私讨论是待办清单，不是设计。
- **保留期只是声明式的。** 存储保留全部记录；类型可声明保留窗口，但没有任何机制执行它。
- **一致性受物化边界约束。** 粗粒度视图只反映到它上次物化为止的记录；其后写入的条目不在其中。我们把它当作**待测量的显式属性**（§5），而不是已解决的问题。
- **规模有限。** 报告的语料是约 2 个月、数千条记录、少数几个类型。本文不展示「需要自适应调整粒度链」的那种规模下的行为。

9. 结论

我们呈现了忆时塔——一个基于 MCP 的时间分层外部记忆服务，其贡献是：(1) **双轨回想模型**，把服务端不调 LLM 的精确回想与依赖总结的语义回想区分开，并把双侧的 token 账明确化；(2) 摘要是**一等公民的时序条目**，支持递归、非破坏的压缩；(3) **类型级粒度链**，适配每个记忆通道自身的频率分布；(4) 把记忆做成**标准化的 MCP 能力**，跨工具共享。我们的主张不是生物保真、也不是任何单一组件在算法上的新颖——每个组件都借自数据库、日常汇总实践或 LLM 记忆系统——而在于**组合本身**，以及「导航 vs 压缩」之间诚实的成本核算。这正是我们提出供讨论、并计划在未来实证验证的贡献。

参考文献

1. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., Gonzalez, J. E. *MemGPT: Towards LLMs as Operating Systems.* arXiv:2310.08560, 2023.
2. Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., Bernstein, M. S. *Generative Agents: Interactive Simulacra of Human Behavior.* UIST 2023 / arXiv:2304.03442, 2023.
3. Zhang, N., Yang, X., Tan, Z., Deng, W., Wang, W. *HiMem: Hierarchical Long-Term Memory for LLM Long-Horizon Agents.* arXiv:2601.06377, 2026.
4. Yang, W., Kan, B., Li, S., Li, L., Qin, Y., Li, J., Bogdan, P., Thomason, J. *RaMem: Contextual Reinstatement for Long-term Agentic Memory.* arXiv:2606.22844, 2026.
5. Zhu, W., Chen, X., Wang, Z., Wang, J., Dong, X., Huang, M., Cai, R., Sang, H., Wang, H., Qiu, P., Deng, Y., Tiwari, P., Rappazzo, B. H., Wang, Y. *AriadneMem: Threading the Maze of Lifelong Memory for LLM Agents.* arXiv:2603.03290, 2026.
6. Zhang, B. *Beyond RAG: LLM Wikis as Living Semantic Memory: Patterns, Empirical Findings, and Open Problems.* Zenodo, 2026. DOI: 10.5281/zenodo.20078453.
7. Miteski, S. *Memory as Metabolism: A Design for Companion Knowledge Systems.* arXiv:2604.12034, 2026（概念 DOI: 10.5281/zenodo.19501651）。
8. Model Context Protocol specification. https://modelcontextprotocol.io

9. Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., Zhang, Y. *A-MEM: Agentic Memory for LLM Agents.* NeurIPS 2025 / arXiv:2502.12110.
10. Chhikara, P., Khant, D., Aryan, S., Singh, T., Yadav, D. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory.* arXiv:2504.19413, 2025.
11. Letta（原 MemGPT）：stateful agents with long-term memory. https://github.com/letta-ai/letta
12. Karpathy, A. *LLM Wiki: A pattern for building personal knowledge bases using LLMs.* GitHub Gist, April 2026. https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f
13. Bell, P. *Compiled Memory vs Governed State: Karpathy's LLM Wiki and the Reflexive Laboratory.* Zenodo, 2026. DOI: 10.5281/zenodo.19507593.
14. Bell, P. *Managed Memory Is Still Not Governed State: Production Lessons from LLM Wiki v2.* Zenodo, 2026. DOI: 10.5281/zenodo.19545947.

AI 使用披露

本文由人类作者与 AI 系统协作完成。**DeepSeek V4.1 Flash** 作为 **AI 共同作者**署名，这与 AiraXiv 预印本平台允许 AI 科学家署名的政策一致；该系统对本文的起草、结构组织与修订，以及 §3 所述实现，均有实质贡献。

人类作者主导了这项工作，亲自执行并核实了 §6 报告的运营测量，逐条核对了全部事实陈述与文献引用，并对内容负全责。文中描述的线上部署行为，观察均来自人类作者自己的系统。

对于不允许 AI 署名的平台（如 arXiv），应改为按本节的「使用披露」形式记录 AI 贡献，而非列为作者。